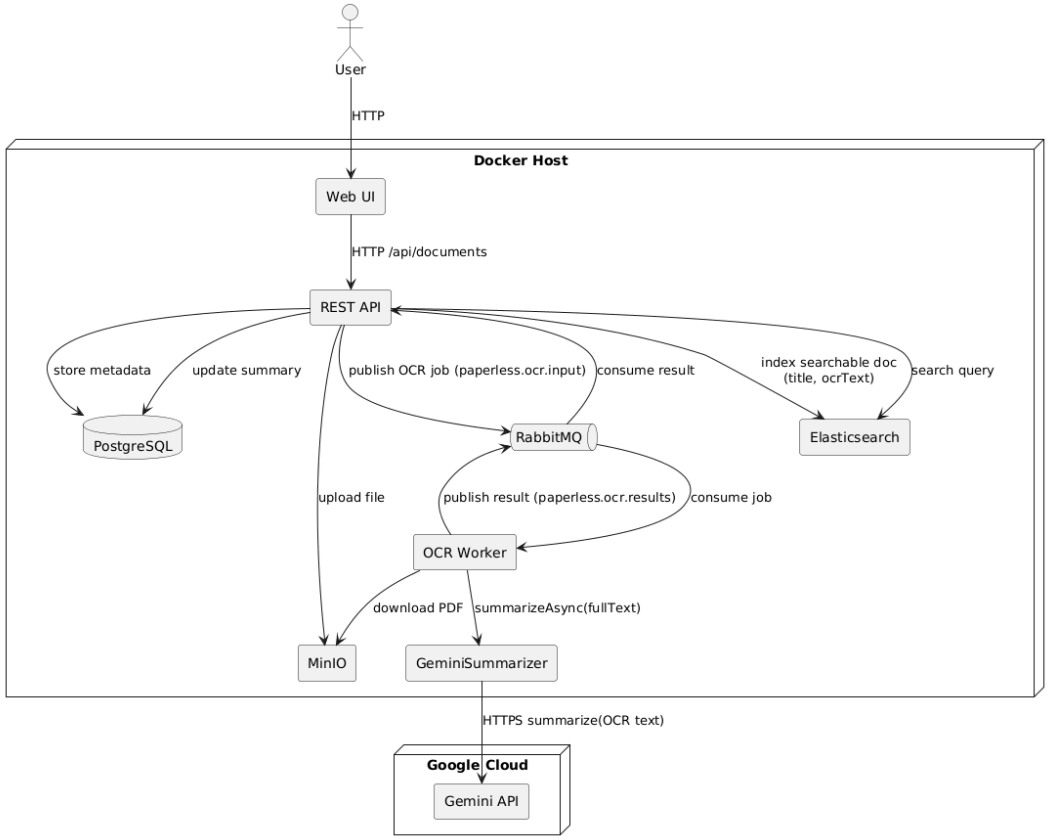
Architecture (Sprints 1–6)
All diagrams can also be found in the diagrams folder

1. **Components (as of Sprint 6):**

- Frontend (Quasar app in frontend):
  - Runs in container paperless-frontend behind nginx web server
- REST API (paperless backend):
  - Handles document upload and retrieval
  - Runs in container paperless-backend
- PostgreSQL:
  - Persists document metadata and other entities
  - Runs in container paperless-postgres
- RabbitMQ:
  - Message broker for document processing
  - Runs in container paperless-rabbitmq
- MinIO
  - object storage for uploaded PDFs
  - Runs in container paperless-minio
- OCR-worker
  - Processes document-upload messages from RabbitMQ
  - Fetches PDFs from MinIO and runs OCR via Tesseract
  - Calls Gemini-based summarisation on the extracted text
  - Publishes the resulting summary back to RabbitMQ
  - Runs in container paperless-ocrworker
- GeminiSummarizer
  - Calls Google Gemini HTTP endpoint for AI summarisation
  - Runs in container paperless-geminisummarizer
- ElasticSearch
  - Full-text search over document fields (Title, Summary, OCR text).
  - Rest API indexes to ElasticSearch

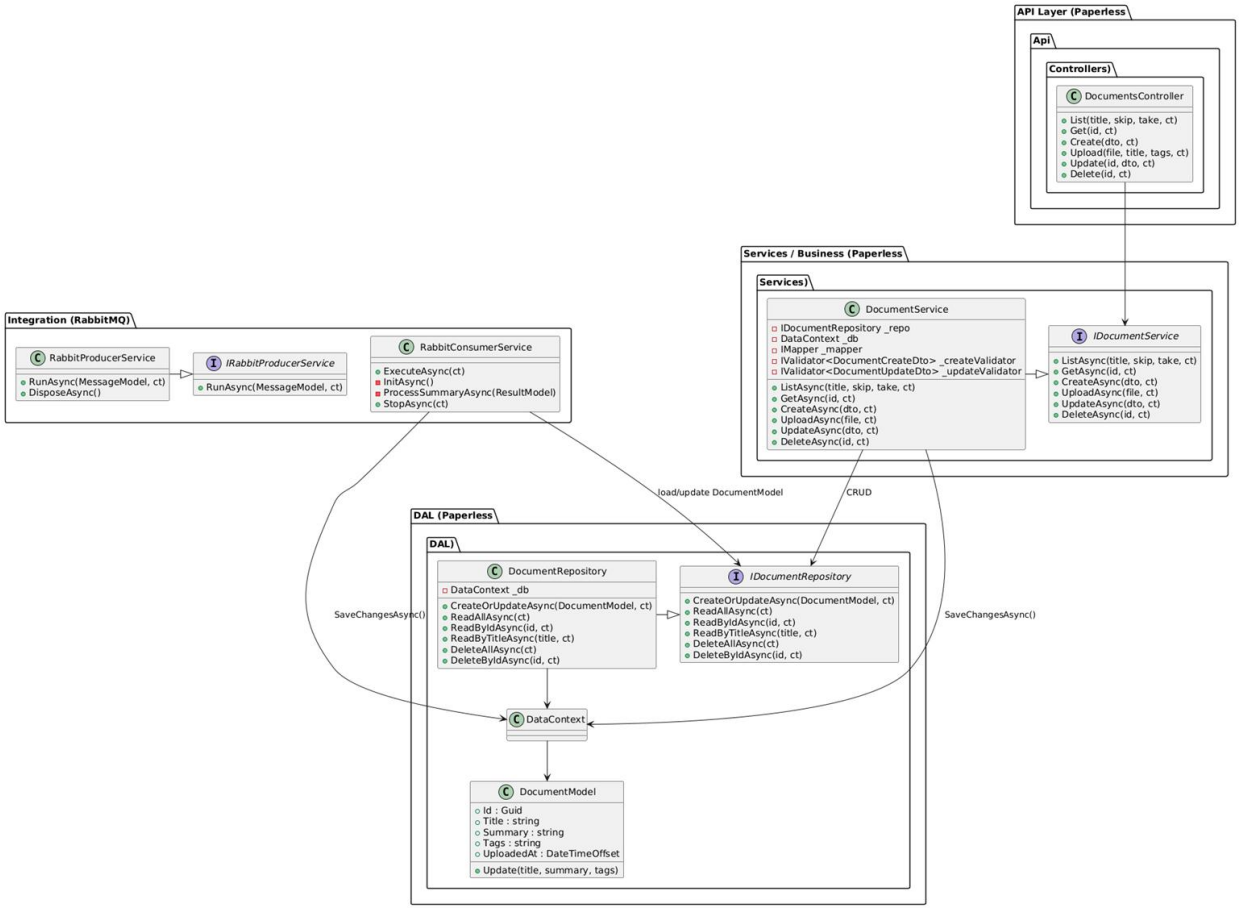


2. **Layered Architecture (Backend)**

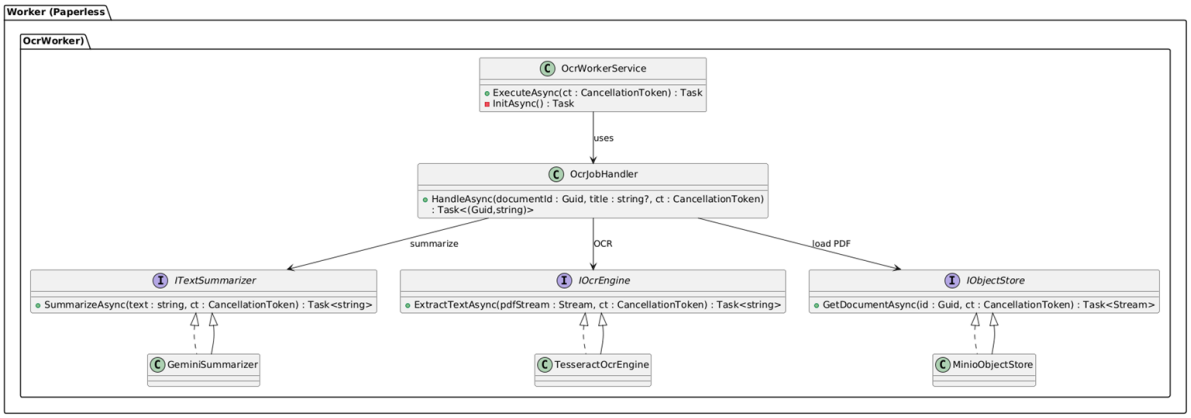
- API Layer
  - Controllers: DocumentController
- Application / Business Layer
  - Services:
    - DocumentService
- Infrastructure / Data Access Layer
  - Repositories: DocumentRepository
  - DbContext: DataContext
- Integration Layer
  - Queue publisher: RabbitProducerService
  - Queue consumer: RabbitConsumerService

3. Class Diagrams

. Backend + RabbitMQ integration



. **OCR worker**

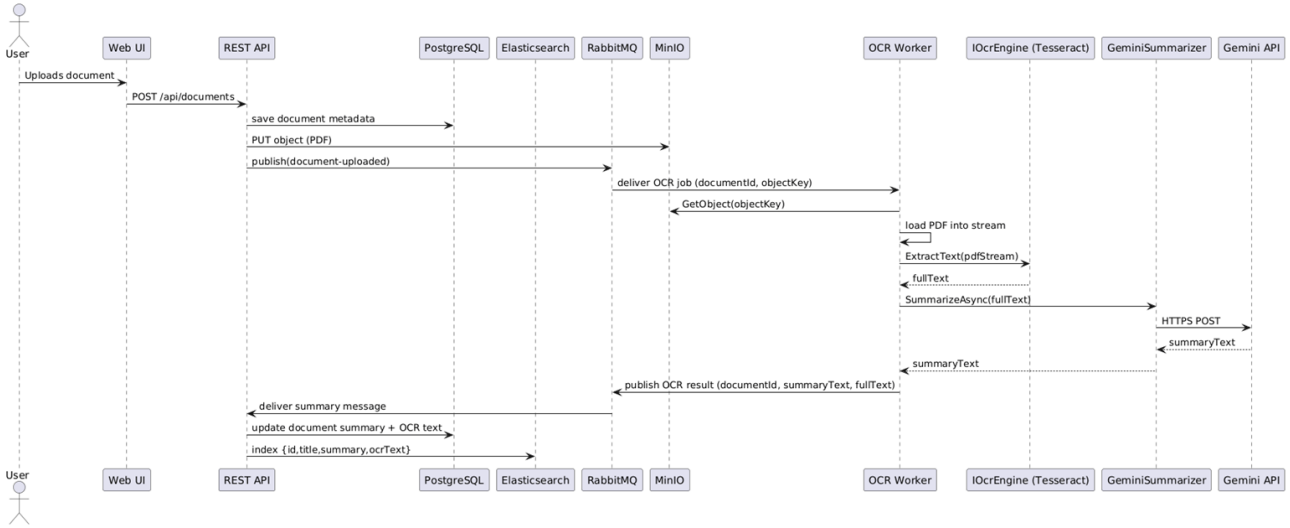


4. Sequence Diagram – Upload Document

Sequence for POST /api/documents/upload:

Actors: **User → Web UI → REST API → RabbitMQ → OCR Worker → MinIO → OCR Worker → GeminiSummarizer → OCR Worker→ REST API**

1. Frontend sends POST /api/documents with file/metadata.
2. API validates request.
3. API calls DocumentService.CreateDocument(dto).
4. Service calls DocumentRepository.Add(document).
5. Repository saves via DbContext to PostgreSQL.
6. API saves PDF in MinIO
7. Service / API publishes a message via DocumentQueuePublisher.
8. RabbitMQ receives message.
9. OCR worker receives message
10. OCR Worker receives document from MinIO
11. OCR worker extracts the text using IOcrEngine
12. OCR worker sends request to GeminiSummarizer
13. GeminiSummarizer sends HTTPS request to Gemini API
14. Gemini Summarizer sends the summary result back to OCR worker
15. OCR worker publishes summary to RabbitMQ
16. RabbitMQ receives message with summary from OCR worker.
17. REST API receives message with generated summary
18. API calls Repo to update DB entry with summary
19. API indexes Title, Text, Summary, and Tags at ElasticSearch


User
Web UI
REST API
PostgreSQL
Elasticsearch
RabbitMQ
MinIO
OCR Worker
IOcrEngine (Tesseract)
GeminiSummarizer
Gemini API
Uploads document
POST /api/documents
save document metadata
PUT object (PDF)
publish(document-uploaded)
deliver OCR job (documentId, objectKey)
GetObject(objectKey)
load PDF into stream
ExtractText(pdfStream)
fullText
SummarizeAsync(fullText)
HTTPS POST
summaryText
summaryText
publish OCR result (documentId, summaryText, fullText)
deliver summary message
update document summary + OCR text
index {id,title,summary,ocrText}
User
Web UI
REST API
PostgreSQL
Elasticsearch
RabbitMQ
MinIO
OCR Worker
IOcrEngine (Tesseract)
GeminiSummarizer
Gemini API